

Laboratory 09

Dual-Core AMP: Messaging Unit & Shared SRAM

Real-Time Operating Systems • M.Eng. ECE • KMUTNB

Platform	NXP FRDM-MCXN236 (ARM Cortex-M33 @ 150 MHz)
RTOS	FreeRTOS v10.6
Toolchain	ARM GNU Toolchain (arm-none-eabi-gcc) + Make
Estimated time	4–5 hours
CLO mapping	CLO 5 & CLO 6

1. Objectives

By completing this lab you will be able to:

1. **Boot** both CM33 cores of the MCXN236 from a single project using SYSCON boot address registers.
2. **Exchange** 32-bit messages between cores using the Messaging Unit (MU) mailboxes.
3. **Implement** a shared SRAM ring buffer with correct DMB memory barriers.
4. **Measure** inter-core IPC latency (MU send → ISR entry) using DWT timestamps on both cores.
5. **Compare** MU-direct vs. ring-buffer throughput for high-rate data transfer.

2. Prerequisites

- ☐ Lab 06 complete (DWT cycle counter working).
- ☐ ARM GNU Toolchain and pyocd working.
- ☐ MCUXpresso SDK for MCXN236 (for `fs1_mu.h` and multicore startup code).
- ☐ Understanding of memory barriers (`__DMB()`, `__DSB()`).

3. Background

3.1. MCXN236 AMP Configuration

The MCXN236 has two independent Cortex-M33 cores (CM33_0 and CM33_1). In AMP mode:

- Each core runs its own independent firmware image with its own FreeRTOS instance.
- CM33_0 boots first (primary core). It writes the CM33_1 boot address and releases the reset.
- Communication uses the **Messaging Unit (MU)** — four 32-bit TX/RX registers per direction, with hardware flow control.
- Shared data lives in **SRAM4** (0x2008_0000, 128 KB) — accessible by both cores.

3.2. Memory Barriers

The Cortex-M33 has a weakly ordered memory model. Without barriers, the CPU may reorder stores before the MU send, causing the consumer to read stale data:

```

1  /* Producer -- on CM33_0 */
2  shared_buf[idx] = data;    /* store */
3  __DMB();                  /* all preceding stores visible before MU
   send */
4  MU_SendMsg(MUA, 0, idx);   /* notify consumer */
5
6  /* Consumer ISR -- on CM33_1 */
7  uint32_t idx = MU_ReceiveMsg(MUB, 0);

```

```

8  __DMB();                               /* MU read complete before we read
   shared_buf */
9  process(shared_buf[idx]);

```

4. Project Structure

```

lab09_dual_core/
  cm33_0/                                <- Primary core firmware
    Makefile
    linker_cm33_0.ld                     <- SRAM0+1 private, SRAM4 shared
    src/
      main_cm33_0.c                       <- boots CM33_1, creates tasks
      adc_producer.c/.h                   <- ADC sampling task (1 kHz)
      shared_mem.h                         <- shared ring buffer declaration
      FreeRTOSConfig.h
      uart.h / uart.c
  cm33_1/                                <- Secondary core firmware
    Makefile
    linker_cm33_1.ld                     <- SRAM2+3 private, SRAM4 shared
    src/
      main_cm33_1.c                       <- MU init, FreeRTOS start
      fir_consumer.c/.h                   <- FIR filter task
      shared_mem.h                         <- same shared ring buffer header
      FreeRTOSConfig.h
      uart.h / uart.c
  flash_dual.sh                           <- flashes both images

```

5. Part A — Boot Both Cores

5.1. Step 1 — Release CM33_1 from CM33_0

In `cm33_0/src/main_cm33_0.c`, after hardware init and before starting the scheduler:

```

1  #include "fsl_syscon.h"
2
3  void Boot_CM33_1(void)
4  {
5      /* Write CM33_1's vector table address (= start of its flash
   region) */
6      SYSCON->CPBOOT    = 0x00080000U;
7      SYSCON->CPUCTRL |= SYSCON_CPUCTRL_CPU1RSTEN_MASK; /* assert
   reset */
8      SYSCON->CPUCTRL &= ~SYSCON_CPUCTRL_CPU1RSTEN_MASK; /* release
   reset */
9      uart_printf("[CM0] CM33_1 released at 0x00080000\r\n");
10 }

```

5.2. Step 2 — CM33_1 Startup

CM33_1's `main_cm33_1.c` initialises its own hardware and announces its presence:

```

1  int main(void)
2  {
3      BOARD_InitHardware_CM33_1();
4      uart_printf("[CM1] Core 1 running at %lu Hz\r\n",
5                  CLOCK_GetFreq(kCLOCK_CoreSysClk));
6      /* ... task creation ... */
7      vTaskStartScheduler();
8  }

```

✓ Checkpoint A

UART showing both [CM0] and [CM1] startup messages.

? Question A1

CM33_0 releases CM33_1 by writing CPBOOT and toggling CPU1RSTEN. What would happen if CM33_0 wrote CPBOOT but never cleared CPU1RSTEN?

6. Part B — MU Direct Messaging

6.1. Step 1 — Send 32-Bit Messages

On CM33_0 (sender task):

```

1 void vMuSendTask(void *pv)
2 {
3     uint32_t count = 0;
4     for (;;) {
5         vTaskDelay(pdMS_TO_TICKS(100));
6         MU_SendMsg(MUA, 0, count);
7         uart_printf("[CM0] sent %lu\r\n", count);
8         count++;
9     }
10 }
```

On CM33_1 (MU receive ISR):

```

1 void MUA_IRQHandler(void)
2 {
3     uint32_t msg = MU_ReceiveMsg(MUB, 0);
4     MU_ClearStatusFlags(MUB, kMU_Rx0FullFlag);
5     uart_printf("[CM1] received %lu\r\n", msg);
6 }
7
8 /* In main_cm33_1.c: enable MU RX interrupt */
9 MU_EnableInterrupts(MUB, kMU_Rx0FullInterruptEnable);
10 EnableIRQ(MUA_IRQn);
```

✓ Checkpoint B

UART showing matching send/receive sequence from both cores.

6.2. Step 2 — Measure MU Latency with DWT

On CM33_0, record the DWT cycle count just before MU_SendMsg. On CM33_1, record it in the first instruction of the ISR. Use a shared SRAM4 variable:

```

1 /* shared_mem.h */
2 typedef struct {
3     volatile uint32_t send_cycles; /* written by CM33_0 */
4     volatile uint32_t recv_cycles; /* written by CM33_1 ISR */
5 } __attribute__((packed)) LatencyTest_t;
6
7 extern LatencyTest_t g_latency; /* in .sram4 section */
```

```

1 /* CM33_0 sender */
2 g_latency.send_cycles = DWT->CYCCNT;
```

```

3 __DMB();
4 MU_SendMsg(MUA, 0, 0xDEAD);
5
6 /* CM33_1 ISR -- first two lines */
7 g_latency.recv_cycles = DWT->CYCCNT;
8 __DMB();

```

After a few exchanges, CM33_1 prints the latency:

```

1 uint32_t latency = g_latency.recv_cycles - g_latency.send_cycles;
2 uart_printf("[CM1] MU latency = %lu cycles (%.1f ns @ 150 MHz)\r\n",
3             latency, latency / 150.0f);

```

✓ Checkpoint B2

Terminal showing measured MU latency in cycles and nanoseconds.

? Question B1

The MU TX register has hardware flow control — MU_SendMsg blocks until the remote core reads the register. For a high-frequency producer (1 kHz), is this acceptable? How would you fix it without blocking the sender?

7. Part C — Shared SRAM Ring Buffer

7.1. Step 1 — Place Ring Buffer in SRAM4

In shared_mem.h (identical copy in both projects):

```

1 #define RING_BUF_SIZE 256
2
3 typedef struct {
4     volatile uint32_t write_idx;
5     volatile uint32_t read_idx;
6     volatile uint32_t missed; /* overflow counter */
7     int16_t data[RING_BUF_SIZE];
8 } __attribute__((packed, aligned(4))) RingBuf_t;
9
10 extern RingBuf_t g_ring; /* placed in .sram4 section */

```

In the linker script cm33_0/linker_cm33_0.ld:

```

MEMORY {
    flash0 (rx) : ORIGIN = 0x00000000, LENGTH = 512K
    sram01 (rwx) : ORIGIN = 0x20000000, LENGTH = 256K /* CM33_0 private */
    sram4 (rw) : ORIGIN = 0x20080000, LENGTH = 128K /* shared */
}

SECTIONS {
    .sram4 (NOLOAD) : {
        KEEP(*(.sram4*))
    } > sram4
}

```

7.2. Step 2 — Producer on CM33_0 (1 kHz ADC Task)

```

1 void vADCTask(void *pv)
2 {
3     TickType_t xLastWake = xTaskGetTickCount();

```

```

4     uint32_t sample_count = 0;
5     for (;;) {
6         vTaskDelayUntil(&xLastWake, pdMS_TO_TICKS(1)); /* 1 kHz */
7
8         uint32_t next = (g_ring.write_idx + 1) % RING_BUF_SIZE;
9         if (next == g_ring.read_idx) {
10             g_ring.missed++;
11             continue; /* buffer full -- drop sample */
12         }
13
14         g_ring.data[g_ring.write_idx] = (int16_t)(sample_count %
15             4096);
16         sample_count++;
17         __DMB(); /* store before index update */
18         g_ring.write_idx = next;
19         __DMB(); /* index update before MU
20             notify */
21         if ((g_ring.write_idx & 0xF) == 0)
22             MU_TrySendMsg(MUA, 0, g_ring.write_idx);
23     }

```

7.3. Step 3 — Consumer on CM33_1 (FIR Filter Task)

```

1     static int16_t fir_h[8] = {1, 2, 3, 4, 4, 3, 2, 1};
2     static int32_t fir_state[8] = {0};
3
4     void MUA_IRQHandler(void)
5     {
6         MU_ReceiveMsg(MUB, 0); /* consume notification */
7         MU_ClearStatusFlags(MUB, kMU_Rx0FullFlag);
8         xTaskNotifyGiveFromISR(xFIRHandle, NULL);
9     }
10
11     void vFIRTask(void *pv)
12     {
13         uint32_t total = 0;
14         for (;;) {
15             ulTaskNotifyTake(pdTRUE, portMAX_DELAY);
16             __DMB();
17
18             while (g_ring.read_idx != g_ring.write_idx) {
19                 int16_t sample = g_ring.data[g_ring.read_idx];
20                 g_ring.read_idx = (g_ring.read_idx + 1) % RING_BUF_SIZE;
21
22                 memmove(&fir_state[1], &fir_state[0], 7 * sizeof(int32_t));
23                 fir_state[0] = sample;
24                 int32_t acc = 0;
25                 for (int k = 0; k < 8; k++) acc += fir_state[k] * fir_h[k];
26                 total++;
27             }
28
29             if ((total % 1000) == 0)

```

```

30         uart_printf("[CM1] processed %lu samples missed=%lu\r\n",
31                     total, g_ring.missed);
32     }
33 }

```

✓ Checkpoint C

UART from CM33_1 showing processed sample count and zero missed samples at 1 kHz.

? Question C1

Remove both `__DMB()` calls on the producer side, rebuild, and run. Do you observe incorrect FIR outputs or missed samples? Re-add the barriers before continuing.

? Question C2

The ring buffer uses `volatile` on the indices. Is `volatile` sufficient to guarantee correctness on a multi-core system? What does `volatile` guarantee vs. what `__DMB()` guarantees?

8. Part D — Comparison: MU Direct vs. Ring Buffer

Run both configurations with a 1 kHz producer for 10 seconds. Fill in the table:

Metric	MU direct (Part B)	Ring buffer (Part C)
Max throughput (samples/s)		
Latency per sample (cycles)		
Missed samples at 1 kHz		
CM33_0 CPU load (%)		

Measure CM33_0 CPU load using `vTaskGetRunTimeStats` with `configGENERATE_RUN_TIME_STATS = 1` and DWT as the timer source.

✓ Checkpoint D

Completed comparison table with measured values.

9. Deliverables

#	Item	Details
1	Part A UART	Both core startup messages
2	Part B2 UART	MU latency in cycles + nanoseconds
3	Part C UART	10 seconds of CM33_1 processing, zero missed
4	Linker script excerpt	<code>.sram4</code> section placement
5	Part D comparison table	Filled with measured values
6	Written answers	Questions A1, B1, C1, C2
7	<code>shared_mem.h</code>	Ring buffer struct listing

10. Key Concepts Demonstrated

Concept	Where you saw it
AMP dual-core boot via SYSCON	Part A
MU direct 32-bit messaging	Part B
IPC latency measurement with DWT	Part B
Shared SRAM ring buffer	Part C
DMB memory barriers	Part C
Throughput vs. latency trade-off	Part D

11. Reference

Resource	Relevant sections
NXP MCXN236 Reference Manual	Chapter MU (Messaging Unit), Chapter SYSCON
NXP MCUXpresso SDK	<code>fs1_mu.h</code> , multicore examples
ARM Cortex-M33 TRM	§A3.4 Memory ordering
Barry, <i>Mastering the FreeRTOS Kernel</i>	<code>xTaskNotifyGiveFromISR</code> , <code>vTaskGetRunTimeStats</code>
